Document number:	P3946R0
Date:	2025-12-14
Audience:	EWG
Reply-to:	*Andrzej Krzemieński <akrzemi1 at gmail dot com>*

Designing enforced assertions

During the 2025 Kona meeting — in response to comment RO 2-056 and the accompanying paper [P3911R0] — the EWG decided to design and add, during the CD ballot period, a new language feature that could colloquially be called "non-ignorable contract assertions". The goal of this paper is to outline the design space and technical trade-offs involved in designing such a feature, based on the SG21's experience with designing the contract assertions.

Goals

The goals for the new feature is to be able to **guarantee** from a function **declaration level** that the function will not proceed upon the indicated incorrect input.

This guarantee is not affected by the compiler switches and the uncertainty of the semantic selection based on configuration and linker technology.

This allows the separation of the code for addressing the incorrect function usage from the positive business logic, so that they can be declared by two different people.

Conceptual model

A high-level programming language, such as C++, is not only for telling what programs should do instruction-by-instruction (imperative aspect), but also for *designing* the program parts (declarative aspect). Contracts, and therefore contract assertions, are for modeling. In this spirit we need to be able to say what precondition assertions communicate (as opposed to what code they generate).

One of traditional and popular models for preconditions is reflected in the statement, "the behavior is undefined unless the condition is true." This implies that when you define what happens when the condition is false, it is no longer a precondition.

In order to accommodate the always-enforcing assertions, we need to provide an alternative model that still maintains the spirit of the above definition. A very good fit is the one in [P2795R5]: erroneous behavior. The way we can teach this is to say:

> A precondition assertion represents a condition that when false upon the function call indicates a bug in the program: something that is encouraged to be reported by tools, such as static analyzers. What happens later, depends on many conditions, one of them being whether we have an enforcing assertion or a fully configurable assertion.

Thus, a function can give a guarantee as to what happens when some condition is false, but this still can be recognized as a bug.

Design criteria

We propose the following criteria according to which the design of the new feature can be evaluated.

1. Function declarations need to be parsable by humans, succinct.
2. The primary purpose of the assertion (communicating what is considered a bug) should be reflected more prominently than the secondary purposes (what happens upon a buggy call).
3. It should not be easy for someone who means to put an enforcing assertion to accidentally put a fully configurable assertion.

Design decisions

The syntax

Variant A

```
void fun(int number, int index)
  pre (number > 0)
  pre! (index >= 0)
  pre! (index < size());
```

Variant B

```
void fun(int number, int index)
  pre? (number > 0)
  pre! (index >= 0)
  pre! (index < size());
```

Variant C

```
void fun(int number, int index)
  pre_maybe (number > 0)
  pre_always (index >= 0)
  pre_always (index < size());
```

Variant D

```
void fun(int number, int index)
  pre (number > 0)
  pre<enforce> (index >= 0)
  pre<enforce> (index < size());
```

Discussion

The most contrived corner cases of variant A (also applicable to variant B):

```
void fun(vector& vec1, vector& vec2)
  pre!(vec1.empty())   // typo?
  pre(!vec2.empty());  // or intended?

void container::fun()
  pre not(empty());  // container not empty?
```

There is another risk associated with variants A and B. One of the requests from SG22 (C Liaison) was that contract assertions should have the form of function calls: `pre(cond)`. This is to enable easier interoperability with C: until C standardizes proper contract assertions, they can define no-op function-style macros for `pre`, `post` and `contract_assert`. It is not clear how relevant that criterion is to WG21. The CD currently sort-of provides it except:

1. It no longer works when attributes are used: `pre [[gnu::safe]] (cond)`.
2. It would stop working for the planned extensions, such as labels.

In variant C, suffixes `_always` and `_maybe` dwarf the most important part of the annotation: the predicate.

Variant D is compatible with [P3400R2], but it makes the fully configurable flavor too attractive (short to spell) over the enforcing flavor. Also, it may be impossible to put [P3400R2] literally as proposed into the Standard, and it may end up being:

```
void fun(int * p)
  pre<std::enforce> (p != nullptr);
```

or

```
void fun(int * p)
  pre [[=enforce]] (p != nullptr);
```

and then we would get a mess rather than compatibility.

Variant B has two positive aspects:

1. It does not make any flavor the default: you always have to choose, and the choice takes very little to type.
2. As a side effect, it solves an old problem of the name `assert`, which we always wanted to use, but could not because of the C function macro `assert()`:

```
assert? (i != 0);  // OK, the configurable flavor
assert! (i != 0);  // OK, the enforcing flavor
assert (i != 0);   // OK, good old macro
```

However, this bakes forever the decision that unsufixed `pre` will never be a thing.

Fixed semantic versus any enforcing semantic

The very minimum that is required from the NSA-safety perspective is a guarantee in the source code that:

1. The predicate in the assertion is evaluated at least once.
2. The control will not proceed to executing instructions guarded by the assertion.

Implementing this minimum still allows certain implementation-defined aspects of behavior. The most prominent one is the choice between `enforce` or `quick_enforce` evaluation semantics.

Note that using semantic `enforce` calls the handler, which may be configured to throw an exception. This still guarantees the non-evaluation of the protected code, but does not guarantee the program termination.

So we have to be very clear what the goal of enforcing assertions is:

1. Not executing the protected code?
2. Not executing any code (unconditional termination)?

Changing other aspects of enforcing assertions

Because enforcing assertions are now decoupled from configurable assertions, there is a temptation to apply to them different design decisions for the controversial aspects, such as:

1. Evaluating the assertion more than once.
   (For instance, it seems conformant to evaluate the assertion twice: first with `observe` semantic, then with `enforce` semantic: we still guarantee not going past assertion.)
2. Treating every variable name as `const` inside the predicate.
3. Translating the throw from a predicate into a contract violation.
4. Allowing side effects in the predicates.

While these decisions are controversial, applying different design choices for enforcing assertions than for configurable assertions would be an even bigger controversy. Should [P3400R2] be adopted in the future, we would end up with two notations for doing almost the same thing:

```
void f (int x)
  pre! (p(x));
  pre<enforcing> (q(x));
```

but still a little bit different.

That said, all the examples of the hardened preconditions use very simple expressions, so if other use cases for enforcing assertions also use only very simple expressions, it is possible to design them slightly differently. For instance, make them ill-formed if the expression in the predicate is potentially throwing.

Guaranteed single evaluation?

Because of the fixed enforcing semantic, there is a possibility to offer an additional guarantee that the predicate will be evaluated exactly once. This has an advantage that in the case of the reported problem, it is easier to determine what instructions were executed. But it also comes with certain disadvantages:

1. It would require evaluating the precondition assertions in the callee, and therefore would loose the ability to report the violation in the caller's source location. In the end, it is the caller who should be pointed at when a precondition is violated.
2. It would enable and encourage the usages not related to correctness at all, where the programmer is guaranteed that arbitrary, possibly mutating, code is evaluated before and after the function call. While contract predicates apply implicit `const` to all referenced variables, it is still possible to perform changes to the program state.

Always-enforced versus hardened preconditions

Always-enforced preconditions are not the same as hardened preconditions. The latter have two special characteristics:

1. They only enforce in hardened implementations: in other implementations they can be ignored.
2. They are only defined by the Standard and they have a "simple enough" expression: not an arbitrary one.

What the second bullet means is that we do not have to consider the cases of visible side effects, double-evaluation, "constification" and similar, or exception handling. The Standard library predicates do not have side effects or throw exceptions.

Other considerations

Proponents of this new feature refer to the unqualified term "safety" for motivation. Because this term is never defined, it is unclear if always-enforced assertions satisfy all the requirements of the undefined "safety". For instance, a programmer can put a predicate with a narrow contract in the enforcing assertion:

```
void f(int i, shared_ptr<vector<int>> v)
  pre!(i < v->size()); // may cause UB
```

Is it "safe" enough? Or is such a feature still "unsafe"?

References

- [P3911R0] — Darius Neațu, Andrei Alexandrescu, Lucian Radu Teodorescu, Radu Nichita, "RO 2-056 6.11.2 [basic.contract.eval] Make Contracts Reliably Non-Ignorable", (https://isocpp.org/files/papers/P3911R0.html).
- [P2795R5] — Thomas Köppe, "Erroneous behaviour for uninitialized reads" (https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p2795r5.html).
- [P3400R2] — Joshua Berne, "Controlling Contract-Assertion Properties", (https://isocpp.org/files/papers/P3400R2.pdf).
- [P3896R0] — Andrzej Krzemieński, "Design goals for a contract support facility", (https://isocpp.org/files/papers/P3896R0.html).